

The Multi-Speaker Registry

Five VocalTractLab speakers: provenance, adaptation and switching

Frédéric Berthommier

fberthommier@users.noreply.github.com

Version 1.1.0 — September 2026

Licensed under GPL-3.0-or-later

Abstract

covtl-pipeline ships **five VocalTractLab speakers** and makes switching between them a single reversible command: `python install_speaker.py install <name>`. This manual, one of the five manuals of the repository, documents the *speaker registry* (the on-disk source of truth and the four software layers a switch must keep coherent) and the provenance of each speaker: the reference speaker `JD3.speaker`; `s1` and `s2`, built from the public-domain Dresden Vocal Tract Dataset (DVTd) MRI meshes by area-function measurement, anatomy mapping and a two-phase adaptation to the Berthommier COVTL model; and `m01` and `w02`, the official VTL 2.4 speakers made loadable by the value-preserving “Option A” schema patch. Every step is stated with the numbers obtained on the real data, and the numbered defect catalogue (D1–D21) records what the transfers taught us.

Contents

1	Introduction	2
1.1	What a VTL speaker file is	2
1.2	The five speakers	3
2	The registry: switching speakers in one command	3
2.1	The registry as a single source of truth	3
2.2	Four layers, one command	4
2.3	The announcement and the f0 default	4
2.4	Interaction with the language pack	5
3	Provenance I — building s1/s2 from public MRI (DVTd)	5
3.1	The data source	5
3.2	Measuring the area function on a mesh	5
3.3	Scales and anatomy mapping	6
3.4	Fitting the shape repertoire and validating	7
4	Provenance II — adapting a speaker to the COVTL model	7
4.1	Phase 1 — articulatory adaptation	7
4.2	Phase 2 — area-function calibration	8
4.3	The corrected calibrator (v2, defect D15)	8
4.4	Worked example: s2 from end to end	8
4.5	m01/w02 in the registry: native-regime verification	9

5 Provenance III — official VTL 2.4 speakers (Option A)	9
5.1 The problem	9
5.2 Option A: three schema transformations, zero value change	9
6 The defect catalogue D1–D21	10
7 Reproducing the chains	12
7.1 Everyday switching (no external data)	12
7.2 Build a speaker from the DVTD archive	12
7.3 Adapt a speaker to the pipeline	12
7.4 Patch official VTL 2.4 speakers (m01, w02)	12
8 File formats	13
8.1 <code>.speaker</code> (XML)	13
8.2 <code>constants.py</code>	13
8.3 <code>registry.json</code>	13
9 Licensing and provenance	13
10 References	14

1 Introduction

1.1 What a VTL speaker file is

VocalTractLab (VTL, Peter Birkholz, TU Dresden) is an articulatory speech synthesizer: the vocal tract is reduced to 40 tube sections, driven by 19 tract parameters and an 11-parameter geometric glottis, and audio is rendered at 44.1 kHz. A VTL *speaker* is a single XML file (conventionally `NAME.speaker`) that fixes everything speaker-specific:

- the **anatomy**: rest positions, ranges (`min`, `max`) and neutral values of the 19 tract parameters (hyoid, jaw, lips, tongue body/tip and lateral shapes, velum, teeth geometry), plus the palate contour and lip geometry;
- the **glottis models**: static parameters and the 11 control parameters (each with range, neutral value and metadata), plus a repertoire of named *shapes* (`modal`, `breathy`, `creak`, ...);
- a repertoire of named **tract shapes** (`a`, `i`, `s`, ...) — vectors of the 19 tract parameters used as starting gestures.

covtl-pipeline does not read these numbers directly: it synthesizes with the **Berthommier COVTL model**, whose speaker-dependent quantities (the 15 polar-plane projection coefficients c_0, c_1, c_2 per parameter, phoneme targets, glottal presets, `f0_default`, effort bounds) live in a generated Python module, `constants.py`. Producing that module from a `.speaker` file is called *adaptation* (§4).

name	origin	sex	source data	VTL (cm)	f0 default (Hz)
jd3	VTL 2.4 reference	male	official VTL2.4.zip	17.00	102.216
s1	DVTD subject 1	male	MRI, 39 y, 1.85 m	20.04	114.908
s2	DVTD subject 2	female	MRI, 32 y, 1.64 m	15.08	181.008
m01	official VTL 2.4	male	official VTL2.4.zip	—	120.000
w02	official VTL 2.4	female	official VTL2.4.zip	—	232.458

Table 1: The five speakers of the registry. VTL is the mean vocal-tract length over tense vowels (measured on the source data where available); m01/w02 anatomies are kept exactly as distributed (not re-measured).

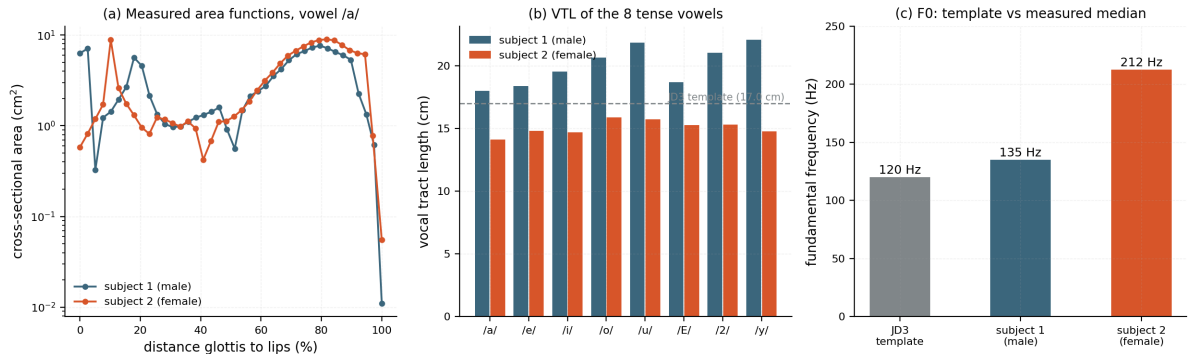


Figure 1: Source-data overview for the DVTD subjects. (a) Measured area functions of sustained /a/ (s1 male, s2 female), glottis to lips. (b) Vocal-tract lengths of the eight tense vowels against the JD3 template (dashed line). (c) F0 templates and measured medians (JD3 120 Hz; s1 135 Hz; s2 212 Hz).

1.2 The five speakers

JD3 is the speaker on which the pipeline was calibrated; it serves as the template for both the DVTD construction (§3) and the adaptation chain (§4), and it is the only speaker that follows the language pack (its vowel targets are re-calibrated per language; `docs/language_pack.pdf`). The four other speakers are shipped with their **native** vowel targets and are validated in that regime.

2 The registry: switching speakers in one command

2.1 The registry as a single source of truth

The directory `vtl_synth/data/speakers/` holds everything:

file	role
<code>registry.json</code>	one entry per speaker: file names, <code>f0_default</code> , description, SHA-256 of both files
<code>ACTIVE_SPEAKER</code>	one-line marker naming the active speaker
<code>jd3.speaker, s1.speaker, ...</code>	the speaker files
<code>jd3_constants.py, s1_constants.py, ...</code>	the constants sources

The design rule: the installed `core/constants.py` is always a *bit-exact copy* of a registry source (its SHA-256 identifies the speaker), while the active *name* is read from the separate marker. `ACTIVE_SPEAKER` is not part of the package: create it once with `python install_speaker.py list`.

2.2 Four layers, one command

A speaker switch must coordinate four layers:

1. **geometry / glottis / f0 default**: the installed `core/constants.py` (swapped file);
2. **orthogonal branch** (VS, VO, TS3, tongue-root context maps): the `.speaker` file resolved through the registry and read at each call;
3. **audio + SVG/video**: the `vocaltractlab-cython` wheel loads its own packaged speaker resource at import time; the installer swaps that resource — the effect applies to the *next* process;
4. **regression baselines**: one file per speaker (`regression_baselines_<spk>.json`), checked by the parameterized test suite and regenerated with `scripts/regen_baselines.py`.

```
python install_speaker.py list           # speakers + active one
python install_speaker.py install s2     # swap constants + wheel resource
python install_speaker.py restore        # back to jd3 (bit-exact)
python install_speaker.py status         # hashes, layer coherence
```

Each `install` runs seven steps: registry lookup, hash check of the source constants, backup manifest (SHA-256), constants copy with post-copy hash verification, `ACTIVE_SPEAKER` marker write, wheel-resource swap with hash verification, and a smoke test in a *fresh subprocess* (synthesis of `a i u` plus a check of the speaker announcement). Any error triggers automatic rollback — the tree is never left half-installed.

2.3 The announcement and the f0 default

Every synthesis announces which speaker each layer uses, e.g.

```
Speaker actif : s2 (DVTD sujet 2 female)  constants=23dd28dd4cf4
              f0_default=181.008 Hz
              geometry: s2  ortho: s2  audio wheel: s2  video: s2
```

and appends explicit warnings when layers disagree (a process started before a wheel swap, or an incomplete install). The CLI resolves the default F0 from the *active* speaker constants (102.216 Hz for `jd3`, 114.908 for `s1`, 181.008 for `s2`, 120.0 for `m01`, 232.458 for `w02`); a forced `-f0` value is announced. Proof on `s2`: `-f0 181.008` measures a median F0 of 183.5 Hz in the output signal.

Baselines are speaker-relative

The end-to-end baselines encode the synthesis output of *one speaker*. `restore -check-baselines` verifies the `jd3` reference bit-exactly after a restore; after any voluntary modification of the signal path (speaker work, language-section changes), regenerate the reference with `scripts/regen_baselines.py`. Per-speaker files keep every speaker checkable independently.

2.4 Interaction with the language pack

The language pack (`setup_lang.py`) edits the managed LANG SECTION (vowel targets, effort gains) *inside* the installed `constants.py`. Its calibration is anchored to JD3: the language-management commands carry a guard that (re)installs jd3 before touching the section, and a speaker swap carries an existing LANG SECTION over to the target constants. The four non-reference speakers are shipped and validated with native targets — keep the default language `en` to work in the native regime. The exact workflows are described in `docs/language_pack.pdf` (§ *Integration and the JD3 guard*) and `docs/expressive_prosody.pdf` (§ *Interaction with the speaker setup*).

3 Provenance I — building s1/s2 from public MRI (DVTD)

3.1 The data source

The **Dresden Vocal Tract Dataset** (DVTD) is a public collection of sustained-phonation MRI of two subjects, published as full-resolution surface meshes.

item	value
Authors	P. Birkholz, S. Kürbis, S. Stone, P. Häsner, R. Blandin, M. Fleischer
Reference	Birkholz et al. 2020, <i>Sci. Data</i> 7:255, doi:10.1038/s41597-020-00597-w
Archive	dvtd.zip, 1 129 467 424 bytes (1.13 GB)
DOI	10.6084/m9.figshare.11897187.v1
License	CC0 1.0 (public domain)

Per subject and per sound (22 sustained sounds: German tense/lax vowels, fricatives, laterals, schwa), the archive provides the inner airway surface mesh from glottis to lips, vocal-tract transfer functions (`measured` and `finite-element`), a reference recording of the same phonation, and manually picked anatomical landmarks (essential for subject 1). Because the license is CC0, the derived speakers and all measurements can be redistributed without restriction.

3.2 Measuring the area function on a mesh

The measurement module works directly on the triangulated inner-surface meshes, without re-meshing:

1. **Medio-sagittal plane:** found as the plane of maximal cut area (scan over the lateral coordinate, then refinement).
2. **2D cavity mask:** on that plane, the cavity is recovered by a parity test on the mesh–plane intersection segments (0.5 mm step).
3. **Centerline:** tracked from the glottis end by contiguity through the laryngeal vestibule and pharynx (across velar pinches of width ≤ 8 mm), then horizontally in the mouth region; smoothed and resampled at 40 stations.
4. **Areas:** each station’s cross-section is the intersection of the mesh with the plane perpendicular to the centerline; all loops are summed, so the piriform fossae are included.
5. **Vocal-tract length (VTL):** curvilinear length glottis $\rightarrow$ lips of the centerline.

The runner measures the 22 sounds and writes, per sound, the VTL, 40 areas, palate contour, transfer-function resonances and the median F0 of the reference recording (autocorrelation).

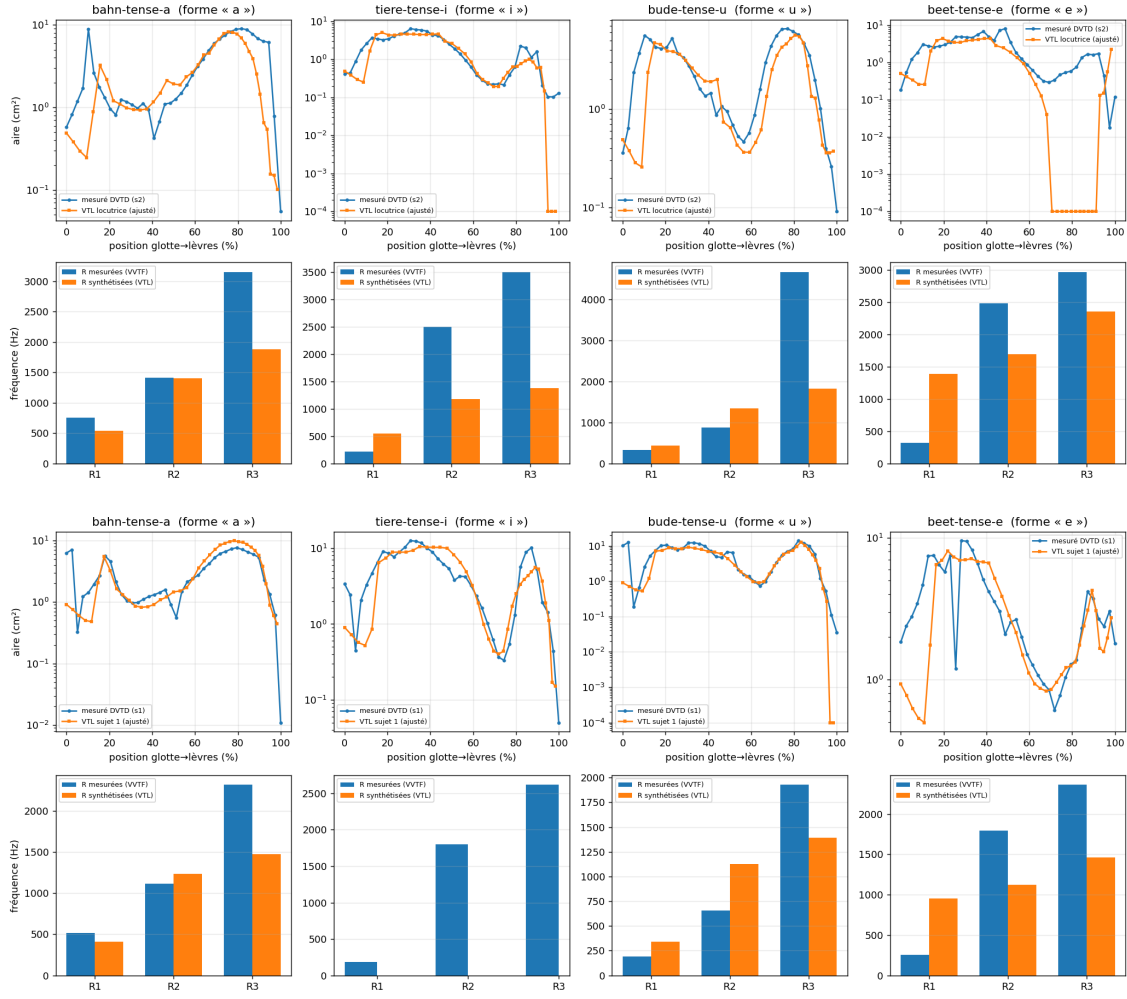


Figure 2: Validation figures for s2 (top) and s1 (bottom): measured vs fitted area functions and measured vs synthesized resonances.

3.3 Scales and anatomy mapping

Two scalar factors drive the mapping onto the JD3 template:

$$s_L = \frac{\text{VTL}_{\text{subject, tense vowels}}}{\text{VTL}_{\text{JD3, same shapes}}} \quad k_{f0} = \frac{f0_{\text{subject, median}}}{120 \text{ Hz}}. \quad (1)$$

	s2 (female, 32 y)	s1 (male, 39 y)
VTL, tense vowels (range)	15.08 cm (14.14–15.89)	20.04 cm (18.03–22.07)
s_L	0.8873	1.1792
F0 median of recordings	212.5 Hz (192–231)	134.9 Hz (115–138)
k_{f0}	1.771	1.1242
Neutral VTL of the built speaker	15.75 cm	18.29 cm

The generator then edits the JD3 template block by block: each block is either *scaled* by s_L (longitudinal anatomy, tongue rest positions, larynx height) or *kept* at the JD3 value (teeth and lips, following the official female-speaker precedent where one exists; jaw rest displacements), with the justification recorded per field in a JSON journal. Subject 1 required one extra check: his 22 meshes do not share a common coordinate frame (three groups of offsets/inclinations); the “lips at y max, glottis at z min” convention was verified sound by sound against the manually

picked lip points, and the measured 20 cm tract was confirmed plausible by an independent 1D transmission-line model (the s1/s2 resonance ratios match the geometric length ratio).

3.4 Fitting the shape repertoire and validating

The built speaker keeps the JD3 shape repertoire as gesture skeleton and refits the free parameters of each target shape so that the *rendered* area function matches the *measured* one (19 tract parameters per shape against the 40-station area function; RMS of log-area 0.57–1.0). Validation compares, per sound, the resonances of the synthesized speaker with the measured transfer-function peaks: mean relative errors for s2 are R1 86%, R2 32%, R3 41% — the first resonance of front vowels is the weakest point, a known limitation of a JD3 skeleton carrying a foreign tongue-shape repertoire. Transferability was checked by replaying JD3 gestures unchanged: formants stay within a few percent for back vowels and diverge on /y/ (F1 +91%), quantifying how far JD3 gestures can be reused.

The chain output is the raw mapped speaker (`speaker_{male,female}_DVTd.speaker`) and, after format normalization, the registry entries `s1.speaker` and `s2.speaker`.

4 Provenance II — adapting a speaker to the COVTL model

A `.speaker` file is not yet usable by the pipeline: adaptation is the deterministic derivation of the `constants.py` module, followed by a geometric calibration.

4.1 Phase 1 — articulatory adaptation

Five stages extract everything derivable exactly from the speaker file:

Stage 1 — the COVTL plane (deterministic). For each of the 15 selectable parameters, the exact 3×3 linear system on the native /a/, /i/, /u/ shape values at the Maeda anchors ($\theta_a = \pi$, $\theta_i = 5\pi/3$, $\theta_u = \pi/3$) is solved for $P(\theta) = c_1 + A \cos \theta + B \sin \theta$, with $c_0 = \sqrt{A^2 + B^2}$, $c_2 = \text{atan2}(B, A)$. Validated on JD3 against the reference constants to $5 \cdot 10^{-7}$.

Stage 2 — closure and contact offsets. Rules transferred from the reference calibration (all verified on JD3 to $< 1.3 \cdot 10^{-4}$): lip-closure LD rule; tongue-tip ceiling TTY (with the +0.15 apical-occlusion bonus of /d, t/); tongue-body raising TCY (+0.25 for the velar occlusion of /g, k/); glottal flutter and aspiration zeroed (except the `hoarse2` preset).

Stage 3 — vowels. /a/ is the anchor $(1, \pi)$; /i/ and /u/ receive a compensation of the TCY raising that reproduces the JD3 targets $\rho_i = 0.60$, $\rho_u = 0.55$; the remaining vowels are least-squares fits on the 15 parameters, ρ clamped to $[0.3, 1]$; the schwa @ is chosen by a multi-criteria area-function cost (no lingual occlusion, limited humping, lip opening $\geq 0.25 \text{ cm}^2$).

Stage 4 — consonants. The 12 contextual place families are fitted by weighted least squares on the Maeda selector; the JD3 calibration deltas of the final closures are then transferred, damped by the amplitude scale of the new speaker (ratio of mean c_0). Derived keys follow fixed rules (`m = b`, etc.).

Stage 5 — glottal source. The 11 glottal presets are copied as-is; `f0_default` is the F0 of the `default` shape (s2: 181.008 Hz; s1: 114.908 Hz) and P_{max} the native maximal pressure ($\approx 8000 \text{ dPa}$).

A *selector invariance diagnostic* (per-family normalized inter-vowel variance of each selector parameter) closes the report; it is the tool of choice to spot parasitic contacts at synthesis.

4.2 Phase 2 — area-function calibration

Phase 2 measures the *rendered* geometry through the VTL API (40 tube areas + articulator codes) and corrects the consonant targets:

- **Stops** (b, d, g_vel, g_pal, dZ, C): ρ set to the measured closure onset (smallest ρ closing all 12 vowel contexts with the required articulator and no parasitic constriction elsewhere) plus a per-place margin (labial +0.36, g_pal +0.35, dZ +0.75, C +0.70; alveolar and velar +0.00). On JD3, measured onsets plus margins reproduce the reference constants exactly.
- **Fricatives** (v, z, Z, D, X): search anchored at the canonical (ρ, θ) minimizing the distance of the gap to its window (e.g. v 0.15–0.25, z 0.10–0.30 cm²), plus a parasitic-constriction penalty, ties broken toward the anchor (place stability).
- **Laterals** (l, L): anchors kept — the central tube does not see the lateral opening (TS3 channel).

4.3 The corrected calibrator (v2, defect D15)

The DVTD transfer test showed that the phase-2 criteria, calibrated on JD3, do not transfer to a foreign anatomy: on s2 the phase-1 anchor of /b/ lands at 280° instead of the JD3-like 50°, where no lip closure exists at all. The shipped calibrator (`scripts/calibrate_fine_v2.py`) applies two fixes:

1. **Stops: two-dimensional onset search.** The search scans θ around the *canonical* place angle ($\pm 60^\circ$, step 10°) in addition to the phase-1 anchor, keeps the smallest- ρ onset, tie-broken toward the canonical place; grid refined from 0.10 to 0.01.
2. **Fricatives: scaled gap windows.** Areas scale as the square of the tract-length ratio, so window bounds are multiplied by s_L^2 (s1: 1.391; s2: 0.787); the grid is centered on the canonical place with local refinement.

A later fix (the “bunching” correction) additionally keeps the native analytic TTY rule instead of the JD3-calibrated stage-2 substitution, which raised the tongue tip at the anchors on the DVTD anatomies; and the schwa is chosen by the multi-criteria cost of stage 3. Result on the shipped constants (v3): no vowel background has a lingual occlusion any more (s2 /a/: $0.00 \rightarrow 0.71$ cm²; all backgrounds ≥ 0.59 cm²).

4.4 Worked example: s2 from end to end

1. **Measure:** 22 sounds; tense-vowel VTL 15.08 cm; F0 median 212.5 Hz $\rightarrow s_L = 0.8873$, $k_{f0} = 1.771$.
2. **Map:** anatomy scaled from JD3 with a per-block decision journal; neutral VTL of the product 15.75 cm.
3. **Fit shapes:** 16 vowels + 6 dvtd-* fricative/lateral shapes fitted on the area functions.
4. **Validate:** synthesized resonances vs measured (R2 mean relative error 32%); JD3-gesture replay transfers within a few percent on back vowels.
5. **Normalize:** schema-normalized speaker file.
6. **Adapt (phase 1):** f0_default 181.008 Hz.
7. **Calibrate (phase 2, v2/v3):** stop onsets re-searched two-dimensionally, fricative windows scaled by s_L^2 ; produces the registered `s2_constants.py` (SHA-256 23dd28dd...).

The s1 run gives $s_L = 1.1792$, `f0_default` 114.908 Hz, constants 47bd805e... After installation, the forced-F0 check `-f0 181.008` measures a median F0 of 183.5 Hz on s2 — the pipeline speaks with the speaker’s own pitch.

4.5 m01/w02 in the registry: native-regime verification

The two Option-A speakers (next section) were admitted to the registry after the same acceptance protocol as s1/s2, run in the native regime (default language `en`):

- `install m01 / install w02`: seven-step swap OK, smoke test SUCCESS, four-layer announcement coherent;
- three runs per speaker (“ba da ga va za la”: 1472 tract frames; “a i u”: 656; “this is easy for us”: 952) all SUCCESS, with median F0 following the speaker preset (m01 124.6/133.1/119.8 Hz around 120; w02 240.3/245.5/232.0 Hz around 232.5) and healthy glottal displacements (the breathy-voice defect D11 is *not* reproduced);
- tract reproduction **bit-exact** against the reference protocol (19 tract columns: $\max |\text{diff}| = 0.000000$ for the three phrases, both speakers);
- per-speaker baselines 16/16 after regeneration (`scripts/regen_baselines.py`); `restore -check-baselines` verifies bit-exactly (15/15 checks, 16 passed).

5 Provenance III — official VTL 2.4 speakers (Option A)

5.1 The problem

The official VTL 2.4 speaker files (M01, W02) target the final VTL 2.4 release, whose geometric glottis is named “Geometric glottis 2025”. The `vocaltractlab-cython` 0.0.13 wheel (the last build installable under Python 3.9 on Windows) bundles an older backend whose `GlottisFactory` knows only three glottis names. Loading M01 crashes the process:

```
terminate called after throwing an instance of 'std::invalid_argument'
what():  [GlottisFactory::makeGlottis()] Invalid glottis name:
        Geometric glottis 2025
```

Three further schema differences break the load silently: a `<velum>` element without `max_nasal_port_area` (read as mandatory by the backend), `<param>` elements without `description/unit` attributes, and the *order* of the anatomy sections (a rebuilt file in the JD3 section order loads where the native order fails).

5.2 Option A: three schema transformations, zero value change

The patch rewrites the speaker XML with three transformations (T1–T3) and *no numeric change whatsoever* (verified by pairing every XML element before/after and comparing all attributes except the three the patch adds or renames):

T1 `<glottis_model type="Geometric glottis 2025|2019">` → `"Geometric glottis"`;

T2 `<velum>`: add `max_nasal_port_area="2.0000"` if absent;

T3 every `<param>`: add `description="..."` and `unit="..."` looked up by name from JD3.speaker.

The backend then interprets the control vector with the historical 11-parameter semantics, so the 2025 parameters are remapped *by index of the 2025 model* (its long names have no homonym in the old model):

2025 idx	2025 long name	historical code
0	f0	F0
1	Subglottal pressure	PR
2	Lower displacement	XB
3	Upper displacement	XT
4	Chink area	CA
5	Phase lag	PL
6	Relative amplitude	RA
7	Pulse shape	PS (matched by meaning, not DP)
8	Flutter	FL
—	(absent from 2025)	DP = 0.05, AS = −10 (JD3 neutrals)

Static parameters are renamed to their historical names *with their values* and must carry a **neutral** attribute (defect D17: without it the old parser warns three times and silently keeps its compiled defaults). The 2025 glottis has no **default** shape: the **modal** shape is duplicated as **default** so that the adaptation stage can read the true neutral phonation (**f0_default**: m01 120.0 Hz, w02 232.458 Hz).

The result is a registry-ready speaker: for M01, the official file (119 299 bytes, SHA-256 438ef897...) → patched file → **m01_mapped.speaker** (195 971 bytes, 9e5598ea...) = the registry’s **m01.speaker**, byte-exact. The alternatives investigated (upstreaming the 2025 glottis classes; rebuilding the backend from the official sources) are documented in the project reports; Option A was chosen because it is local, reversible and value-preserving. Its known limits: synthesis runs the *historical* geometric glottis (the 2025-specific parameters are loaded but their effect is not validated), and the official ZIP ships inverted pressure presets (defect D18: **voiceless-*** at 8000 dPa where JD3/M01 voiceless presets sit at 0) which are transmitted faithfully.

6 The defect catalogue D1–D21

The three transfer campaigns (JD2 as a neutral probe, M01 as a truly foreign speaker, s1/s2 as constructed speakers) produced a numbered defect catalogue. Each entry is a case study: symptom, cause, workaround.

D1 —

Old-format speaker files destroy glottal presets silently (the 2.3 format stores shapes in a section the reader ignores). Fix: detect the format before adaptation — normalization is the mandatory step 0.

D2 —

Copying a **.speaker** into the package data has no effect: the pipeline resolved one hard-coded path and the wheel loads its own resource. Fix: the registry with per-layer resolution.

D3 —

Regression baselines fail after a speaker swap (they encode one speaker’s output hashes). Fix: per-speaker baseline files + **regen_baselines.py**.

D4, D7 —

Rounding and grid noise: three decimals destroy glottal preset precision; phase-1 θ values carry grid noise. Fix: rounding conventions aligned with the reference.

D5, D6, D8, D9 —

Documentation-vs-code drift (JD3-calibrated constants emitted without alerting; inexact docstring; hard-coded glide fallback; process observation). Addressed in the constants generation and the written procedure.

D2-bis/ter —

The 2025-glottis crash family: unknown glottis name; velum/param schema gaps. Workaround: Option A (§5).

D10 —

Phase-2 criteria do not rescale (tube zones and gap windows are JD3-calibrated cm^2 constants). Fix: `calibrate_fine_v2.py` (canonical-place 2D onset search, s_L^2 -scaled windows).

D11 —

No phonatory normalization: a foreign speaker speaks breathily (native glottal displacements 3–4× more open than JD3). Status: open issue; not reproduced on the final m01/w02 constants.

D12 —

The “N/15 convergence” diagnostic is speaker-relative: it only measures distance to JD3. For foreign speakers, internal criteria (effective closures) must replace it.

D13 —

Reference-chain prosody ignores the speaker (f0 contour from an internal template). Consequence: cross-speaker F0 comparisons must force `-f0` or use the pipeline’s own prosody.

D14 —

CLI F0 hard-coded to the JD3 value silently re-pitched every speaker. Fix: the default is resolved from the active speaker’s constants; a forced F0 is announced.

D15 —

“JD3-derived speaker $\Rightarrow$ convergence” is false: the skeleton graft transfers anatomy, not behavior; every closure/gap criterion must be re-measured on the new anatomy (the v2 calibrator).

D16 —

Subglottal pressure can exceed $P_{\max}$ ($\approx 13\,000$ vs $8\,000$ dPa); the adapter’s `EffortBounds` are not enforced anywhere. Open issue (clipping vs rescaling to be decided).

D17 —

Renamed 2025 static glottal parameters need a `neutral` attribute, else the old parser silently keeps compiled defaults. Fixed in the mapping.

D18 —

The official VTL 2.4 ZIP ships inverted pressure presets (`stop/voiced-plosive` at 0 dPa, `voiceless-*` at 8000 dPa). Transmitted faithfully (no value correction); bounded effect, runs healthy. To report upstream.

D19 —

`tract_state_to_transfer_function` returns NaN (wheel 0.0.13) on a zero-area section (closed lips). Workaround: 1D transfer-function method with an area clamp $\geq 1\text{ mm}^2$.

D20 —

The language pack and the baselines interact: a jd3 reference captured under one language regime fails under another. Fix: regenerate the reference after any voluntary change of the language regime; native references are archived.

D21 —

Registry semantics: a speaker swap carries the previous LANG SECTION into the target constants. Correct for JD3 (the pack speaker); for the native-target speakers, install them under the default language `en` (their constants carry no section) so the native regime is reproduced by itself.

Three recurring themes

(i) *Silent* failures of the VTL backend (crashes without exceptions, ignored sections, parser fallbacks) — always verify by hash or by a fresh-subprocess smoke test, never trust a return code alone. (ii) *Calibrated constants travel badly*: anything expressed in cm^2 , degrees or dPa must be re-derived or scaled for a new anatomy. (iii) *Layers drift apart*: a speaker exists in four layers, and only the announcement plus hash manifests keep them coherent.

7 Reproducing the chains

Prerequisites: Python 3.9+, numpy, scipy, vocaltractlab-cython==0.0.13 (cp39, Windows), Pillow, imageio-ffmpeg, cmudict; the DVTD archive (1.13 GB, CC0) and the official VTL2.4.zip for the construction chains. The registry itself needs none of these: the five speakers ship with the package.

7.1 Everyday switching (no external data)

```
python install_speaker.py list
python install_speaker.py install s2
vtl-synth run --phonetic "ba da ga va za la" -o out/
python install_speaker.py restore --check-baselines
```

7.2 Build a speaker from the DVTD archive

```
# per subject: 22 inner-surface STL meshes + VVTF + reference wav
python mesurer_tout.py          # -> mesures_s2.json (s1 variant: _s1)
python generer_speaker.py      # skeleton graft, s_L, k_f0, journal
python ajuster_shapes.py       # fit vowels + dvtd-* shapes
python valider.py              # resonances vs VVTF, transferability
python validation_acoustique_1d.py # independent 1D plausibility model
```

7.3 Adapt a speaker to the pipeline

```
python normaliser_speaker.py SPEAKER.speaker # step 0: schema fix
python adapt_speaker.py     SPEAKER_norm.speaker # phase 1
python calibrate_fine_v2.py SPEAKER_norm.speaker --ref REF.py # phase 2
```

7.4 Patch official VTL 2.4 speakers (m01, w02)

```
# official M01.speaker / W02.speaker extracted from VTL2.4.zip
# expected SHA-256: M01 438ef897..., W02 c4b6c1e7...
python patch_option_a.py      # T1+T2+T3 -> *_pa.speaker
python mapping_glottal_2025.py # -> *_mapped.speaker = registry files
```

8 File formats

8.1 .speaker (XML)

Root `<speaker>`; child `<vocal_tract_model>` containing `<anatomy>` (one `<param>` per tract parameter with `min`, `max`, `neutral`), the palate contour, teeth and lips blocks, `<velum>` (with `max_nasal_port_area`), and `<shapes>` (named vectors of the 19 tract parameters). A second child `<glottis_models>` holds, per model, the static parameters, the control parameters (each `<param>` with `description` and `unit`) and the glottal preset shapes. Section order matters for the 0.0.13 backend; the safest reference is `JD3.speaker`.

8.2 constants.py

Generated module with the COVTL plane per parameter (`CO_VTL`), vowel targets (`VOWEL_TARGETS`), consonant targets (`CONSONANT_TARGETS`) with their selectors, glottal presets, `SpeakerConfig` (`f0_default`) and `EffortBounds` ($P_{\max}$). The installed file is always a bit-exact copy of the registry source.

8.3 registry.json

A `_meta` block (creation date, pristine wheel-resource hash, JD3 constants reference hash) plus one block per speaker: `speaker`, `constants`, `f0_default`, `description`, `speaker_sha256`, `constants_sha256`.

9 Licensing and provenance

The pipeline code is GPL-3.0-or-later. The speaker data stack several origins:

- **JD3, M01, W02** — official VTL2.4.zip (<https://www.vocaltractlab.de>); VocalTract-Lab is free for research and educational use. M01/W02 are redistributed in patched (Option A) form with attribution, the customary practice of the VTL community; the construction chain can rebuild them deterministically from the official archive (download script, expected SHA-256, patch chain) should the author object.
- **s1, s2** — derived from the Dresden Vocal Tract Dataset (Birkholz et al., *Sci. Data* 7:255, 2020), license CC0 1.0: the derived speakers and measurements can be redistributed without restriction.
- Every redistributed data file is inventoried with its SHA-256 in the registry manifest; the generated constants sources are shipped byte-exact, so the hashes recorded in `registry.json` match the files.

10 References

1. P. Birkholz, *VocalTractLab* — articulatory speech synthesizer (API 2.4). <https://www.vocaltractlab.de>
2. P. Birkholz, S. Kürbis, S. Stone, P. Häsner, R. Blandin, M. Fleischer, “Dresden Vocal Tract Dataset”, *Scientific Data* 7:255, 2020. DOI: 10.1038/s41597-020-00597-w; data: 10.6084/m9.figshare.11897187.v1 (CC0 1.0).
3. P. Krug, *VocalTractLab-Python* (vocaltractlab-cython wheel). <https://github.com/paul-krug/VocalTractLab-Python>
4. F. Berthommier, “A mathematical model of the vowel space”, arXiv:2111.00868 [eess.AS], 2021.
5. F. Berthommier, “Why can big.bi be changed to bi.gbi? A mathematical model of syllabification and articulatory synthesis”, arXiv:2307.02299 [eess.AS], 2023.